ECADS

Emergency Services Contact & Dispatch System for offline incident management

Name:	Feargal McLoughlin
Student Number:	03041239
Topic:	Project Final Report
Course:	HDip. Computer Science
Supervisor:	Dr. Sinead O'Neill
Date:	29/03/2026

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student… Fergal Mc Loughlin ..... Date 28/03/26.....

Work Place Mentor…… Date …….

Contents

Table of Figures 5
Table of Tables 6
Glossary 7
Introduction 8
Problem Domain 8
Background 8
SAFE System 9
Issues Identified 9
Proposed Solution: ECADS 10
Design 12
Efficiency in Simplicity 12
Requirements 13
Users 14
Call Taker 14
Dispatcher 15
Supervisor 15
Administrator 15
Use Cases 15
Technical System Overview 16
Components 17
Clients 17
Framework 18
Interface 18
Data Storage 19
Security 19
Application Components 20
Methodology 21
Methodology Chosen 21
Implementation 22
Sprints 22
Managing system development 23
How the system was developed 24
Sprint 1 24
Sprint 2 26
Sprint 3 28

Sprint 4 & 5 30
Meeting Requirements 34
Project Specific Considerations 38
Data Validation 38
Testing 38
Use of AI 39
Conclusion 40
Personal Reflection 40
Areas for further development 42
Reports 42
Merging Data with SQL 42
Windows Forms Application Design 42
Windows Forms - Rich Text Box 43
Final Thoughts 43
References 44
Appendix 1 – Challenge Solutions 46
PowerShell 46
C# 46

Table of Figures

Figure 1 - SAFE - Incident/Contact Record 10
Figure 2 - BASH Implementation 12
Figure 3 - Police Emergency Call Process (SAAB, 2025) 13
Figure 4 - BASH & PowerShell ECADS workflow 14
Figure 5 - ECADS simple system overview - deployed to one client 16
Figure 6 - ECADS deployed to local server overview 17
Figure 7 - Application Components and their properties 20
Figure 8 - ECADS.sh Main Menu 25
Figure 9 - Data validation using regex in Bash 25
Figure 10 - Allowing the user to review data and confirm before updating an Incident 26
Figure 11 - PowerShell Main menu 28
Figure 12 - example of applying colour to stings and not values in PowerShell 29
Figure 13 - Coloured strings and data feedback to end user PowerShell 30
Figure 14 - ECADS Home view 31
Figure 15 - Contact records view 32
Figure 16 - Utilising Closure Codes and MessageBoxes to ensure data is updated correctly 33
Figure 17 - Code to validate if the user wished to progress depending on the result from a MessageBox 33
Figure 18 - Diagnostic capture when running Windows Forms version of ECADS 39
Figure 19 - Method to apply bold on Contact Record or Incident numbers in Display box 41

Table of Tables

Table 1 - Sprints 22
Table 2 - Functionality meeting Requirements across all three iterations 34

Glossary

AI	Artificial Intelligence
AWK	Scripting language used for sorting data
BASH	Born again shell
BCP	Bulk Copy Program
CAD	Computer Aided Dispatch
CLI	Command Line Interface
GREP	Global Regular Expression Print
GUI	Graphical User Interface
OS	Operating System
REGEX	Regular Expressions
SED	Stream Editor
SSO	Single Sign On
UAT	User Acceptance Testing

Introduction

The aim of the project is to demonstrate learning and understanding from studies undertaken in, and to complete, the Higher Diploma in Computer Science at South East Technological University. The discussed project will use different scripting and programming languages and platforms to build a complete and effective data recording system for emergency services to use when there is "network down" scenario. While the world is benefitting from increased online presence, it is critical that emergency services have contingencies to continue with their primary roles of call taking from members of the public and dispatching their resources to incidents even when networks are not available.

This report aims to provide information on the project currently under development and detail the overall vision of the project and how it may be achieved in the timeline provided.

Problem Domain

Background

In November 2020 it was announced that An Garda Síochána (Ireland's National Police & Security Service) had signed a contract with SAAB to provide their dispatch system SAFE. This is the first deployment of this system to a national police force (SAAB, 2020). This €13.5 million announcement was a significant milestone for An Garda Síochána as it had been highlighted by the Garda Inspectorate in 2009 that a new system was needed as the previous system was in operation in excess of 30 years (Garda Inspectorate, 2014 & McCarthaigh, 2021).

SAFE System

The SAFE system is a powerful, truly unified command, control and communications platform which is supported by a significant estate of infrastructure and servers. Without network communication, users are unable to use the SAFE application and enter data. This can happen for many reasons; SQL servers deadlocked, poor network connectivity, server unexpectedly rebooting, etc. The aim of this project is to provide a light offline solution that user can continue to capture data on is there is any network connectivity issues experienced with the main SAFE application.

Issues Identified

In the event of network connectivity issues, or a potential cyber-attack, which may render the SAFE system inaccessible to end users, currently the options available are very limited resulting in end users recording information on a paper based form. Identifying an area for improvement, the proposal to support end users, and to ensure a contingency, this project proposes the development of a simplified data capture application that can be deployed locally on individual clients to allow users continue to capture data in an accurate manner which can be audited.

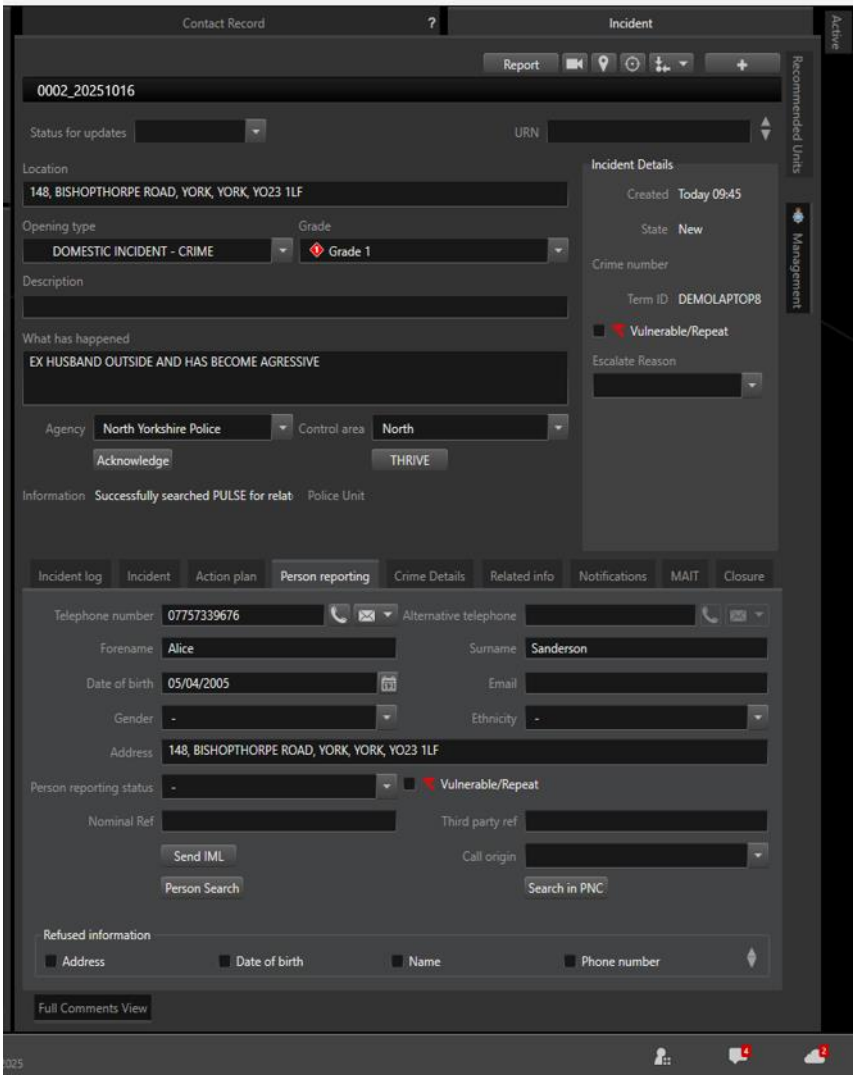

Figure 1 - SAFE - Incident/Contact Record

Proposed Solution: ECADS

This project is broken down into three stages:

1. Create a CLI based application using BASH as a proof of concept

   Rationale:-

   a. In its simplest form, this application should allow operators capture data easily as the interface is consistent with the online system.

b. Data entered should be retrievable and allowed to be reviewed before uploaded to the main database once back online.

2. Create a CLI based application using PowerShell

   Rationale:-

   a. Clients machines run Windows OS and PowerShell whereas BASH is only available through WSL on these clients. The PowerShell script will ensure that this project is theoretically viable for the UAT and Pre-Production environments, if ever needed.

3. Create a Windows graphical application, building on the information and knowledge gained from the previous two iterations of the application

   Rationale:-

   a. A graphical application would be more suited to the method of data capture and current work practices. Where possible the graphical application will resemble the current SAFE application to allow users easily engage when transitioning between application.

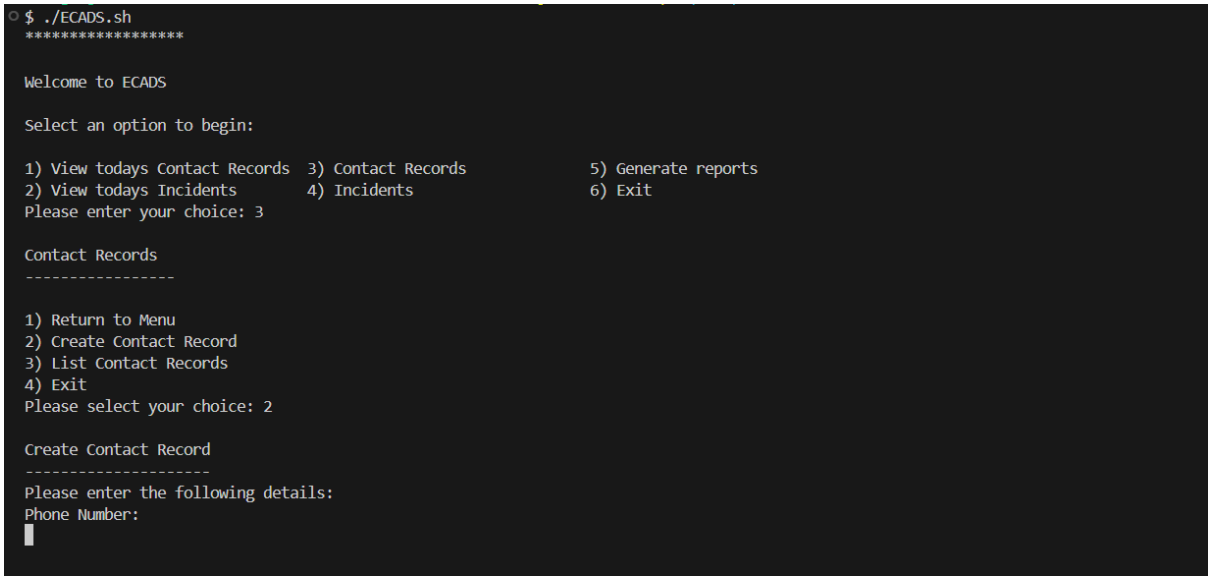

Figure 2 - BASH Implementation

Design

Efficiency in Simplicity

Initially in attempting the project the first consideration was what would be technically feasible to implement. However, on presenting the implementation to an operator for review it was realised that the end user should be the focus. Whilst it is important that functionally the application should work, there is no point in having a functioning application if nobody can use it. Likewise, the scenario is that there is no network connectivity, so the application must be able to, at the very least, work locally on one single client. The application must be able to allow the users follow the Police Emergency Call process as displayed in figure 3. It is this thinking that drove the three stage implementation of the project, from BASH script proof of concept to a proposed graphical application.

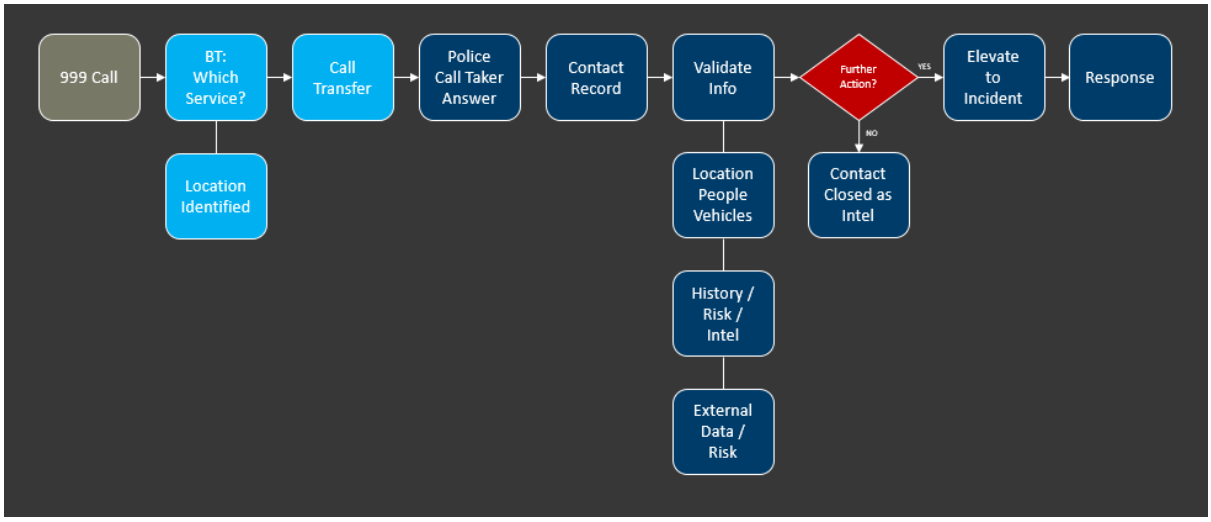

Figure 3 - Police Emergency Call Process (SAAB, 2025)

Requirements

The requirements for ECADS are as follows:

1. Application must work without network connectivity
2. Application must be easily accessible and understandable
3. As users work in a high stress environment, the application must be user friendly
4. The application must require little to no training to use
5. No log in required due to closed network, security handled by a different section
6. A user can create an Incident from the creation of a Contact Record – this incident must have the Contact Record referenced in the Incident details
7. Contact Record for that Incident should be referenced in the Incident
8. Active Incidents must be able to be updated with comments to detail actions taken by users and responders
9. Closed Contact Records and Incidents must not be able to be updated further
10. Contact Records and Incidents created on the system must be able to be retrieved within the system

11. Data entered must be able to sync with the main SAFE database when network connectivity has been restored
12. The application must be able to generate reports to assist Supervisors in their role

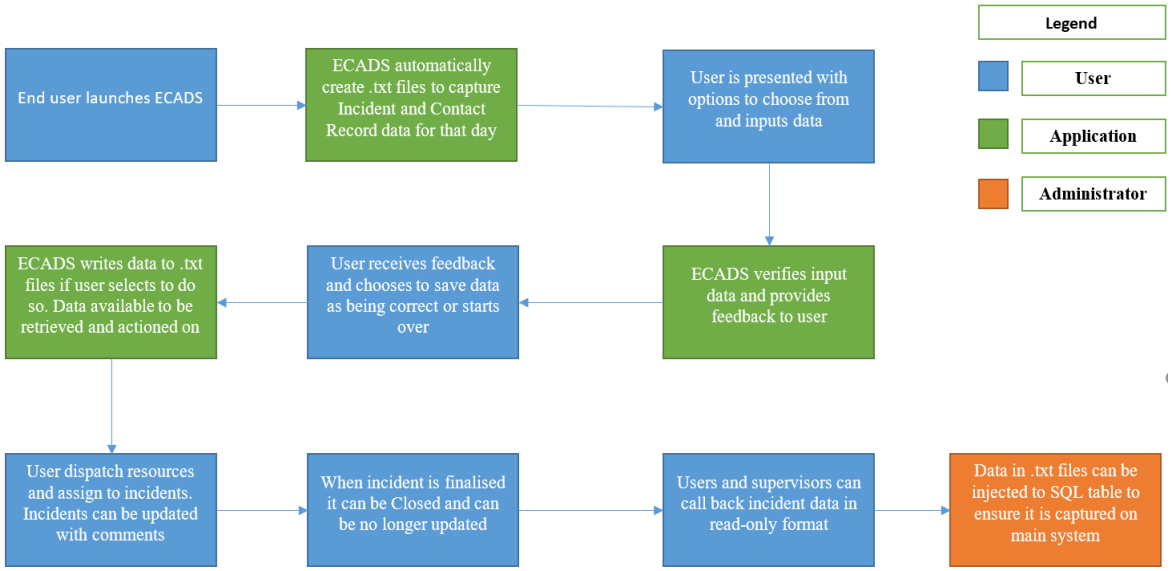

Figure 4 - BASH & PowerShell ECADS workflow

Users

Within each Control Centre there are four distinct roles that staff occupy and perform certain duties under. Each of the four roles will have the same permissions within the ECADS application. It may be beneficial however to understand each of the roles as it may assist in grasping the design of the ECADS system;

Call Taker

A user who answers calls from the public and inputs the data on a Contact Record. They can create Incidents resulting from the Contact Record or independently. (Mainly concerned with creating Contact Records and Incidents)

Dispatcher

A user who maintains contact with emergency response personnel and determines how an Incident should be responded to. They are responsible for dispatching resources to incidents and updating them. (Mainly concerned updating and closing Incidents)

Supervisor

A user who monitors data entry and the number and type of Incidents being created. They have overall responsibility for the Control Centre where the users work. (Mainly concerned with volume of Contact Records and Incidents – reviews data entry and the updating of Incidents in a timely manner)

Administrator

A user who provides technical support for the application. They do not enter data but are responsible for ensuring the data is merged with the main system when it is back online.

Use Cases

The ECADS system must address the issues faced by users when there is no network connectivity to the main GardaSAFE application servers. This can occur for multiples of reasons:

- Server disk space at maximum capacity
- SQL server issues including, expensive queries producing deadlocks on the server
- Major weather events impacting infrastructure and connectivity to data centre sites
- Cyber-attack scenario

ECADS has been designed with these issues in mind and has been developed in a stepped release format staring with a basic Bash script as a proof of concept, through to a fully formed graphical Windows Forms application. The design of ECADS means that the application can be deployed on single user clients or, to scale up, it could be deployed on a local server which the clients can access.

Technical System Overview

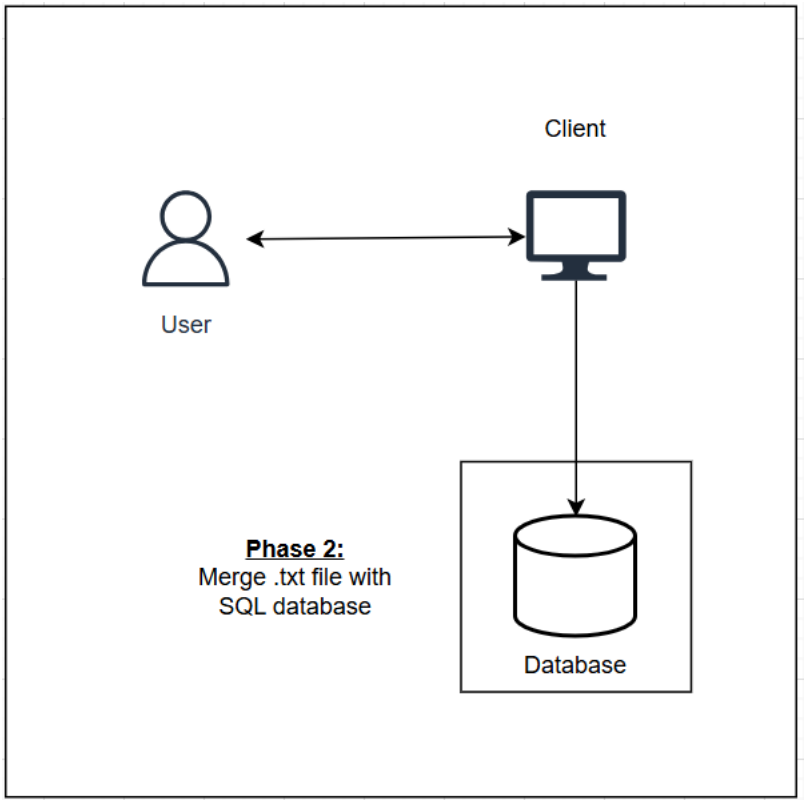

Figure 5 - ECADS simple system overview - deployed to one client

ECADS is a client based application that can either run locally on a client or can be accessed from a shared local server. During Phase 1 of development, a simple application was devised that was able to capture data entered in .txt files with single line entry. All three versions of the ECADS system developed contain the same entities, i.e. objects that exist within the system

and the user can Create, Read or Update. It is not permissible for any user on the main GardaSAFE application to Delete any data and as such this was replicated in the ECADS system.

With some minor changes to configuration, the ECADS system can be deployed to a local server, isolated from the main network and accessible to local clients, if the number of users in the control room would require same.

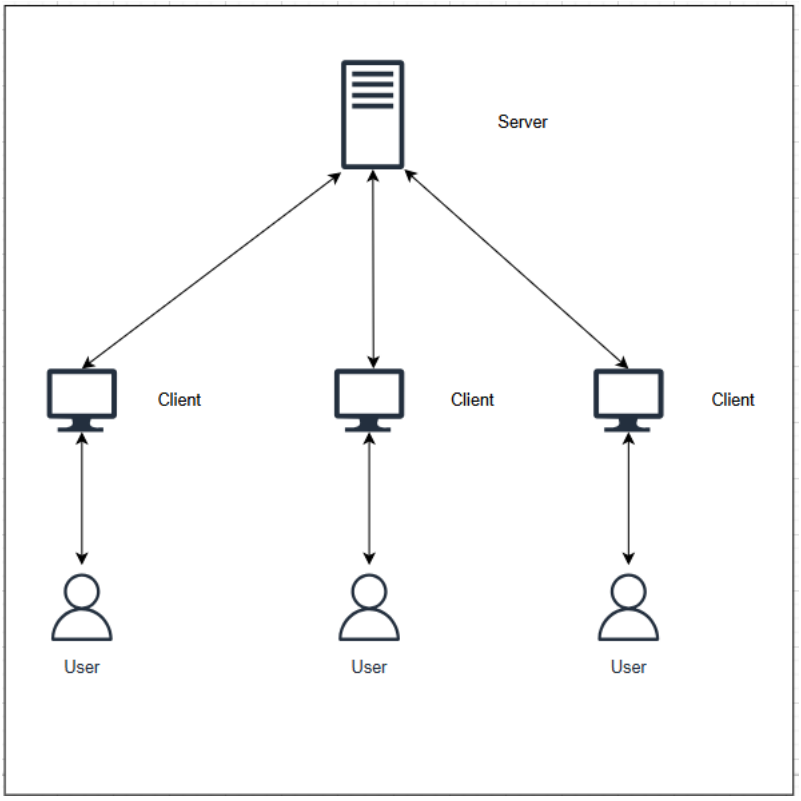

Figure 6 - ECADS deployed to local server overview

Components

Clients

All clients within the control rooms operate on a version of Windows OS. The PowerShell Script and Windows Forms application have both been tested on UAT clients running Windows

10 and Windows 11 and worked without issue. All clients had 16GB of RAM installed with an Intel Core i3 processor.

Framework

The ECADS graphical application is built on the .NET Framework 4.7.2. The .NET Framework is used mainly in the designing and running of Windows applications and web services (Microsoft Learn, 2026.b). Building an application in Microsoft Visual Studio proved to be challenging at times but overall quite intuitive. The .NET Framework was chosen due to all clients running the full GardaSAFE application required it installed. This also allowed for some familiarity of the ECADs system with current users of the main application. The .NET Framework utilises the common language runtime which manages the code for security and robustness (Microsoft Learn, 2026.b).

Interface

The interface of the ECADS system can exists as either a simple command line interface if using the BASH or PowerShell version, or it is a graphical Windows Forms application. The graphical Windows Forms application was the preferred choice following UAT testing and is an easily configurable layout that can meet the demands of users.

The Windows Forms graphical interface allows the users to read data they have entered easily and to also review before submitting for entity creation.

Data Storage

These .txt files exist in three separate entities; a Contact Records .txt file, an Incidents .txt file and an Incident Comments .txt file. By utilising .txt files to store the data, it is easy to manipulate and retrieve same for display back to the user. When designing the storage system I was conscious that this data will need to be merged with the main application data at some point. Having discussed this with a main application system architect, it was agreed that this was achievable by using .txt files which could then by injected using BCP to a staging table in SQL to be merged with the main application data (Microsoft Learn, 2026.a). This was the intention from the outset but unfortunately, although theoretically possible, it was not implemented. The idea of storing the data via .txt file occurred to me when looking at the first iteration of the system, the BASH script. I needed something that was uniformly accepted by any system, particularly when the first iteration was developed on a predominantly unix/linux platform and my solution needed to account for Windows clients. It occurred to me that, if data within a .txt file is divided on the newline break, then really it is just a human-readable array of strings. Each index of the "array" being each line entry in the .txt file.

A .txt file is universally accepted by any operating system or platform and when read from a program, it can by styled in varying was to address the needs of users. The size of the data is negligible when stored in such a format which is another bonus.

Security

Security is handled by another department within an Garda Síochána and the application will be deployed to production clients which reside within an isolated network. Only users with administrative privileges on the network can access the C: drive on production clients and the final Windows Forms application is coded to create the relevant .txt directories and folders on

the C: drive within the ProgramData folder, which is inaccessible to standard users. To log in to a client, users must provide their username and password, from there they are authenticated and the main applications become available for use vis Single Sign On (SSO).

Application Components

The application at its core is quite simple and consists of a user who can interact with the application to create three distinct "entities"; Contact Records, Incidents and Incident Comments. Contact Records and Incidents can exist independent of each other, but an Incident must exist for an Incident Comment to be created. As such, Contact Record and Incident details can be recalled on their own for viewing, but Incident Comments can only be recalled when viewing the details of an Incident.

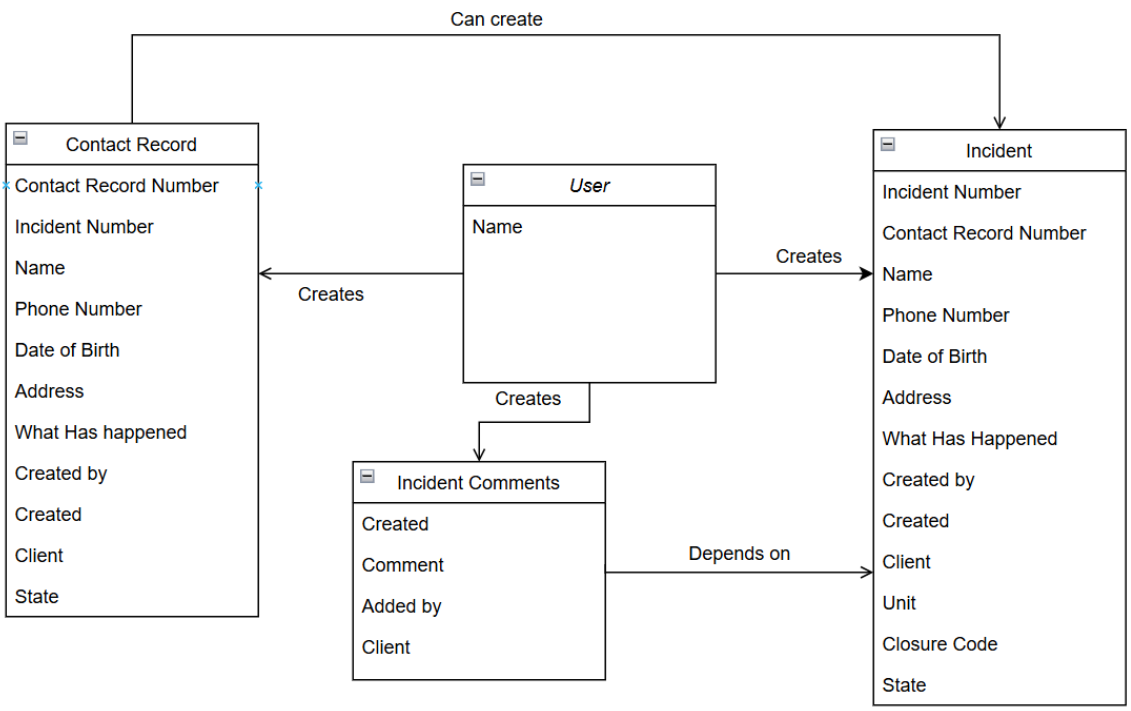

Figure 7 - Application Components and their properties

Although the “user” does not exist as an application entity, the user is referenced in each of the entities by name and that is derived from the signed in user on the computer running the application. Details of the application components and their associated properties are detailed in Figure 7.

Methodology

Methodology Chosen

Having worked in ICT the past four years, and the previous two years in an e-learning capacity, the methodologies that I am most familiar with came to the fore for this project. For the design and implementation methodology I chose the agile approach.

Working with an Agile methodology really means being flexible and responsive to the needs of the user and being able to develop to meet changing needs. It cannot be better phrased than “agile is a group of methodologies that demonstrate a commitment to tight feedback cycles and continuous improvement” (Atlassian, 2026). Personally, I feel that this approach is very beneficial to both customers and developers alike due to the interaction between the development team and the end user, plus it allows for on-the-fly changes or development to take place. This can make development more efficient and streamlined as end-users/customers get a feel for what is being developed and can feed into it throughout the development process and not just at the start with rigid requirements.

There are some drawbacks to using the agile approach to development, however. With a focus on production and development, it can at times fall foul of a lack of documentation. As the speed of development can increase with an agile approach, it also has the opportunity to

introduce errors that are not always easily caught. As I had access to users, both of the current GardaSAFE system and of its' predecessor, I used them as my "customers" which allowed me to feedback on development and have them test what was already implemented.

The predecessor to GardaSAFE was a simple text based application which managed incidents capture and dispatch via input text commands. As a throwback to that system and as a proof of concept where I felt I could trial my design ideas in a simple format, I decided first on implementing ECADS as a BASH script. This was a fortunate choice as it afforded me the opportunity to quickly develop an "application" where I could visualise many concepts that would form part of the final solution. In doing this I was engaging with the agile methodology as I could easily identify issues and change my development strategy to work around issues and build on increased functionality. By understanding what the "customer" or end-user needed the system to do, I was then free to develop innovative was to achieve that functionality.

Implementation

Sprints

To manage the time to produce the three iterations of ECADS I divided my time up into sprints of c. 2 weeks. Allowed me to focus on each iteration in turn, developing the iteration to meet the requirements and also to address any issues encountered with some time for reflection and feedback from users.

Table 1 - Sprints

Sprint	**Date**	**Comment**
1	26 January 2026	Start of BASH Script

2	09 February 2026	User testing of BASH Script and research of PowerShell
3	25 February 2026	Start of PowerShell Script
4	08 March 2026	Start of Windows Forms application
5	23 March 2026	Continued work on Windows Forms application and report writing

Managing system development

In order to keep version control and to develop this application in a methodical format, I created a git repository and remotely updated it on Github. This allowed to make regular commits and also review previous versions of code implementation when I was faced with challenges to overcome later on in development. I found the use of Github very beneficial as I was using three different coding languages (Bash, PowerShell and C#) and although I was able to overcome all issues, the manner in which I did was not as obvious in each language. An example of this was reading the .txt files and filtering for specific patterns. I utilised some form of regex in each of the three languages, but found that more often than not, Bash was the most intuitive and had the easiest way to implement the functionality, whereas complexity increased with PowerShell and C#. This may also be as a result of my initial familiarity with Bash and my lack of knowledge of PowerShell and C#. Nevertheless, Github allowed me to quickly review how I overcame the issue in Bash and lead me to ways of thinking of how I could overcome the same issues in both PowerShell and with C#.

How the system was developed

Two IDEs were used during the development of this project; Visual Studio Code and Microsoft Visual Studio. I found that Visual Studio Code was very easy to navigate and very easy to switch between the Bash terminal and PowerShell terminal making running the scripts very simple for testing and debugging when new aspects were added to the code.

I found Microsoft Visual Studio 2026 a little difficult to engage with at first, particularly with merging the Git repository, but this was soon overcome and I would attribute it to lack of familiarity with the IDE. The development was broken down into 5 sprints of approximately 2 weeks' duration where each of the iterations were taken in turn and focused on. This allowed the development of a concept piece and also highlighted challenging areas early on to avoid them clogging up the development work line at a future date.

Sprint 1

Sprint 1 consisted of focusing on the initial proof of concept phase of the development of the project. This consisted of designing a Bash script that had all the main functionality, albeit in a linear CLI based format. Having identified what was required for the system to provide a meaningful functionality to users and the organisation, I focused on developing out each entity with the basic information required. This scripted version was menu driven and consisted of three distinct Bash scripts; ECADS.sh, contactRecord.sh and incident.sh. ECADS.sh acted as the main menu of the scripted application from which the other two scripts could be called. This allowed the user to have a very easy way of navigating the application by returning to the main menu whenever necessary and selecting their desired option.

```
******************

Welcome to ECADS

Select an option to begin:

1) View todays Contact Records  3) Contact Records          5) Generate reports
2) View todays Incidents        4) Incidents                6) Exit
Please enter your choice:
```

Figure 8 - ECADS.sh Main Menu

Viewing of Contact records and Incidents was achieved by reading the .txt files that contained the saved data and displaying them back to the user. Data validation and identifying closed Incidents to not allow further update was achieved using grep, regex and awk to discern what entities had been closed and could no longer be updated.

```
        while true; do
        echo "Phone Number: "
        read phoneNumber
#Check to see if the input for phoneNumber was empty
            if [ -z "$phoneNumber" ]
            then
            echo "Invalid input! Please enter a phone number"
            continue
            fi
#Check to see if the input for phoneNumber contains only digits, if it contains anything but digits then provi
            if ! [[ "$phoneNumber" =~ ^[0-9]+$ ]]
            then
            echo "Only numbers are accepted as input for phone number, please try again"
            continue
            fi
            break
        done
```

Figure 9 - Data validation using regex in Bash

There were many challenges encountered during this initial development; creating accurate date variables, capturing the data entered in a way it could be easily retrieved, developing unique ID's for both Contact Records and Incidents that matched the exact manner in which

they are created on the actual Production application of GardaSAFE. Each of these were overcome with some thought and different perspectives on how they might be achieved. With the rigidity of how the unique ID's must display it forced me to think in different ways to determine the correct solution. This phase of the development also taught me the value of data input and feedback from the application to the user, e.g. stopping the user from inputting text characters for a phone number and then allowing the user review what data they had input and to affirm if it was correct before submitting for recording to file. This was very obvious as the system is linear and the user did not have opportunity to amend a value there and then like in a graphical application with data input fields.

```
echo -n "Are these details correct? (y/n):"
read answer
if [[ "$answer" == "y" || "$answer" == "Y" ]]
then
lineNumber=${closeIncident:0:4}
echo $lineNumber
sed -i "${lineNumber}s/$/ Closure Summary: $closureSummary Closure Code 1: $closureCode1 Unit: $unit A
echo "Added successfully"
else
echo "Aborting process"
fi
source incident.sh
;;
```

Figure 10 - Allowing the user to review data and confirm before updating an Incident

Sprint 2

With the conclusion of Sprint 1, I was in the privileged position of being able to engage with current and former users of the online GardaSAFE application and sought their assistance in putting the script through UAT testing. This provided invaluable information and feedback on what had been developed to date, what worked well and what needed to be improved upon. The first major challenge I faced with this sprint was getting the users access to the Bash script version of the application. All Dev, UAT, Pre-Production and Production environments I had to work with operate on Windows OS and the Bash script was only available if I installed

GitBash or the clients ran WSL. The second option was not available but I was able to install GitBash on a select number of clients to allow users interact with and test it. This highlighted the very next step in development; I would need to develop the application in a format that was compatible with the clients that were in use by the organisation. To make it simple and have ease of access for users when needed, I determined that the next iteration of the application would be created using PowerShell.

Users commented that the linear format of the Bash script was quite simple to use, they did find it repetitive. It was noted that certain aspects could be improved upon, outside of the menu driven format, there was no distinction regarding the data for the end-user, it was all white text on a black background. This was very interesting to me and formed part of my research for how to address this with the PowerShell version of the application. Aside from the restrictions on the users with the Bash script, the feedback was positive to the extent they had thought it better than recording the data in writing on a bespoke sheet. Supervisors noted that when Incidents were closed that they could no longer be updated as a positive and that it addressed the need for governance when engaging with the system.

I used the rest of this sprint to get familiar with the PowerShell scripting language and attempted to try and make a GUI application with PowerShell. These attempts were outside of what I committed to my Github repository as they were explorative in nature. Although I was able to display a window with some text on it and a button, I found that this was quite difficult and in the end abandoned this idea as it really would not be up to scratch for a Production environment. As a result of this research I decided I would replicate and enhance what I had created in Bash via PowerShell. This meant that if the application was needed in a hurry, a simple PowerShell script could be deployed, it would work on the clients and would at least address some of the governance needs of the organisation.

Sprint 3

Sprint 3 commenced on the 25th of February and I was very optimistic going into the sprint, having decided on the scope. PowerShell was a new scripting language to me and, although I had a very good scaffold in the Bash script I had created, I did find it difficult to get to grips with some of the syntax. The first change I made was to combine the three previous Bash scripts into one PowerShell script. This was achieved by creating four separate functions in the PowerShell script, one each for displaying a menu, Contact Records, Incidents and a Main function. The show-menu, contactRecords and incidents functions simply displayed menu options to the end user while I utilised the main function to engage a switch statement that would call each of the required functions. This allowed the application to be all contained within the one script.

```
==================== Main Menu ================
1: View todays Contact Records
2: View todays Incidents
3: Contact Records
4: Incidents
5: Generate reports
6: Exit
===============================================
Please enter your choice:
```

Figure 11 - PowerShell Main menu

The major advantage of scripting in PowerShell was that it was very easy to incorporate colour to strings when providing information back to the user. This was done with being somewhat imaginative with the Write-Host command as I wished to have the title of the value in colour but not the value itself, to distinguish it. I found that by writing the value in a separate Write-

Host command directly after the value title which had the colour applied and by utilising the –NoNewline argument that it worked seamlessly.

```
Write-Host "Processing..."
Start-Sleep -m 500
Write-Host
Write-Host "The data you have entered is: " -ForegroundColor Green
Write-Host
Start-Sleep -m 500
Write-Host "Phone Number: " -ForegroundColor Yellow -NoNewline
Write-Host $phoneNumber
Write-Host "First Name: " -ForegroundColor Yellow -NoNewline
Write-Host $firstName
Write-Host "Surname: " -ForegroundColor Yellow -NoNewline
Write-Host $surName
Write-Host "Person Address: " -ForegroundColor Yellow -NoNewline
Write-Host ($arrPersonAddress)
Write-Host "What has happened: " -ForegroundColor Yellow -NoNewline
Write-Host $whatHasHappened
Write-Host
```

Figure 12 - example of applying colour to stings and not values in PowerShell

The major disadvantage I discovered while developing the application in PowerShell was I did not have the use of awk, grep or sed which I had used in Bash to overcome some issues. I managed to use other commandlets in PowerShell to achieve the same result but I certainly found the code to be more verbose. Bash scripting was more straightforward but again, this may just be an example of lack of familiarity with the syntax and getting to know the language. This disadvantage was not something I felt was a negative however. Certainly it slowed down my development of the script, but it made me tackle the issue in different ways and challenged me to solve the problem in a manner that would achieve the best result.

Once again, I passed this version through UAT calling on colleagues to test. There was positive feedback on the display of data now and the ease of deployment was a major plus. What was obvious however is the users wanted a GUI application rather than a CLI based one.

```
Create an Incident
------------------
Please enter the following details:
Phone Number: 0897787765
First Name: John
Surname: Smith
(To complete entering an addess, press 'Enter' without inputting any text)
Please enter address: 123 Main Street
Please enter address: Tuam
Please enter address: Galway
Please enter address:
What has happened: Report of a stolen lawn mower at this address
Processing...

The data you have entered is:
Phone Number: 0897787765
First Name: John
Surname: Smith
Person Address: 123 Main Street Tuam Galway
What has happened: Report of a stolen lawn mower at this address

(Note: Incident number will be automatically generated when data is submitted)
Are these values correct? (y/n):
```

Figure 13 - Coloured strings and data feedback to end user PowerShell

Sprint 4 & 5

Sprint 3 finished without the development of any reporting capability, similar to that of the Bash iteration, I had focused on getting the main functionality working so I could learn and then progress forward. Sprint 4 focused on developing out the Windows Forms application iteration. This would be the final iteration as I had very little time to develop further plus it was the iteration that I felt would address all requirements and incorporate all feedback from users.

I had never coded in C# prior to this and I had also never used Microsoft Visual Studio. There was a steep learning curve for me as I found the jump from PowerShell to C# quite large, but I was thankful I had taken the time with PowerShell as some of the syntax was quite similar.

I started out by devising how I would display the different entities and also how I would capture the data for them. I reverted to what I had done previously with the Bash script and treated each entity in its own right, this time instead of a separate script for each entity I had 5 separate forms; Home, Contact Records, Incident and Reports, with a fifth form ECADS to act as a host form.

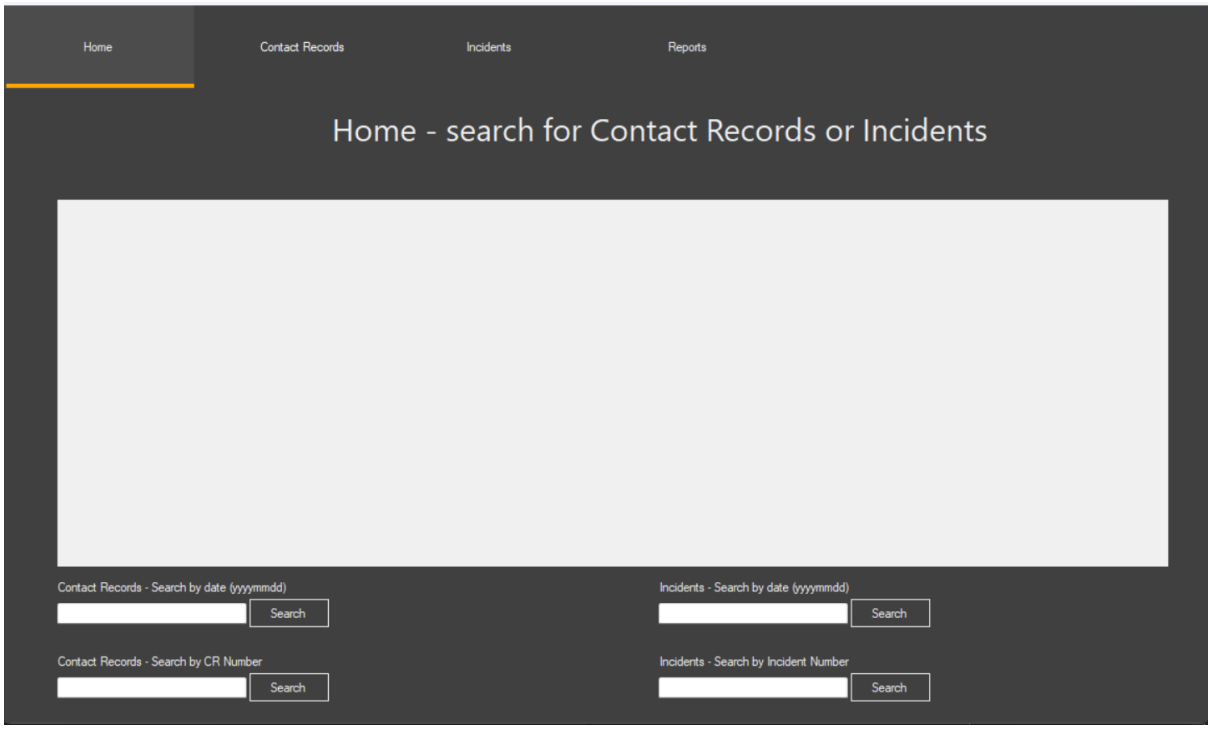

Figure 14 - ECADS Home view

This was somewhat of a mistake. It meant that I was now contained to using this bespoke way of displaying the information and I discovered quite quickly that this limited my options for user functionality. As an example to explain this further, I wished that on the Contact Records view, when a user chose to create and Incident from a Contact Record, that the Incidents view would open automatically. As the Contact Record view is contained within its own form, triggering an “on-Click()” function on a button on a different form is not so simple. I was able to call the function on the other form and get it to action, however it would only show on the

creation of a new form. I then tried to revert my change of having separate forms for each part and created bespoke user-controls within the application. This could have worked if I had implemented the form view differently but I had progressed too far at that stage. Ultimately I left this functionality out and the application works fine, however upon further research, there is a dedicated tabs control already and if I had used this control, I could have implemented what I wanted with ease.

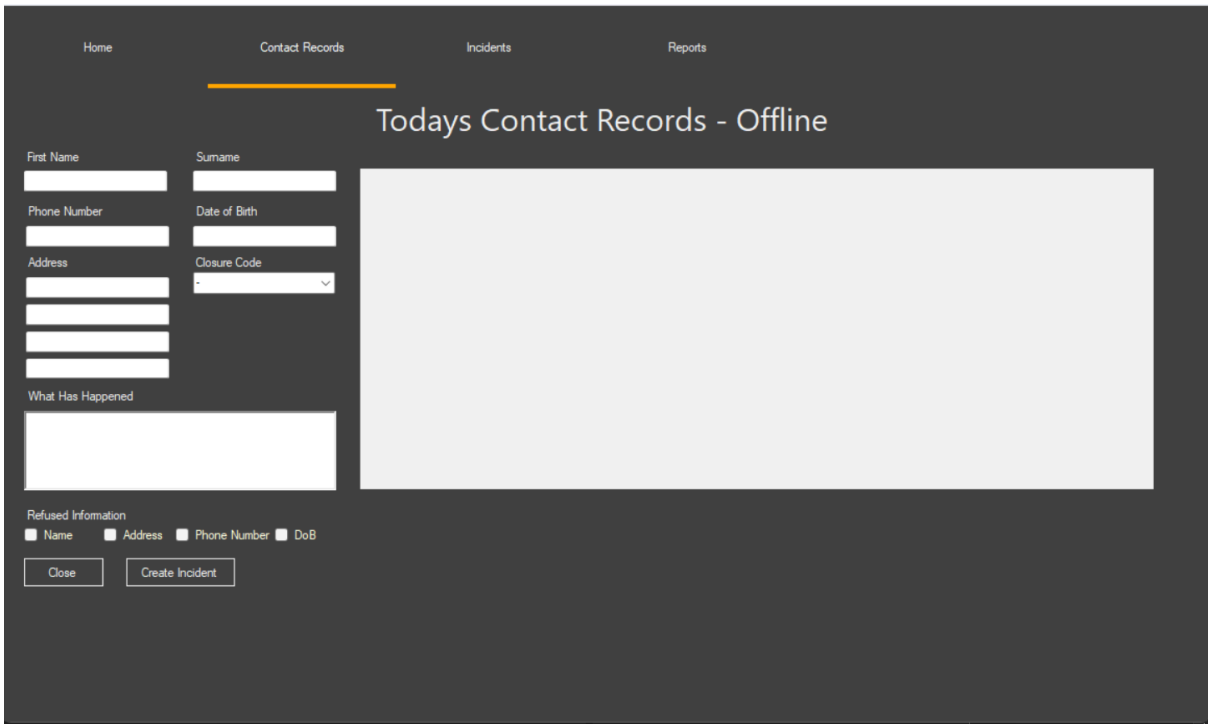

Figure 15 - Contact records view

The major advantage of building the application with Windows Forms was the ease of design and the intuitive way I could add code to the application using click events and by tailoring the different text fields with tab options. The graphical view of data lets the user review what they have typed in before committing to file. The commits to file are also "guarded" with the use of conditions before any commit is made. By utilising Closure Codes, something which exist in the GardaSAFE application, I was able to leverage different functionality, e.g. the Closure Code of Create Incident when creating a Contact Record would allow for the auto-creation of

an Incident, or by selecting a Closure Code for an Incident, this would then allow progress to determine if the user actually wanted to close the incident they had selected.

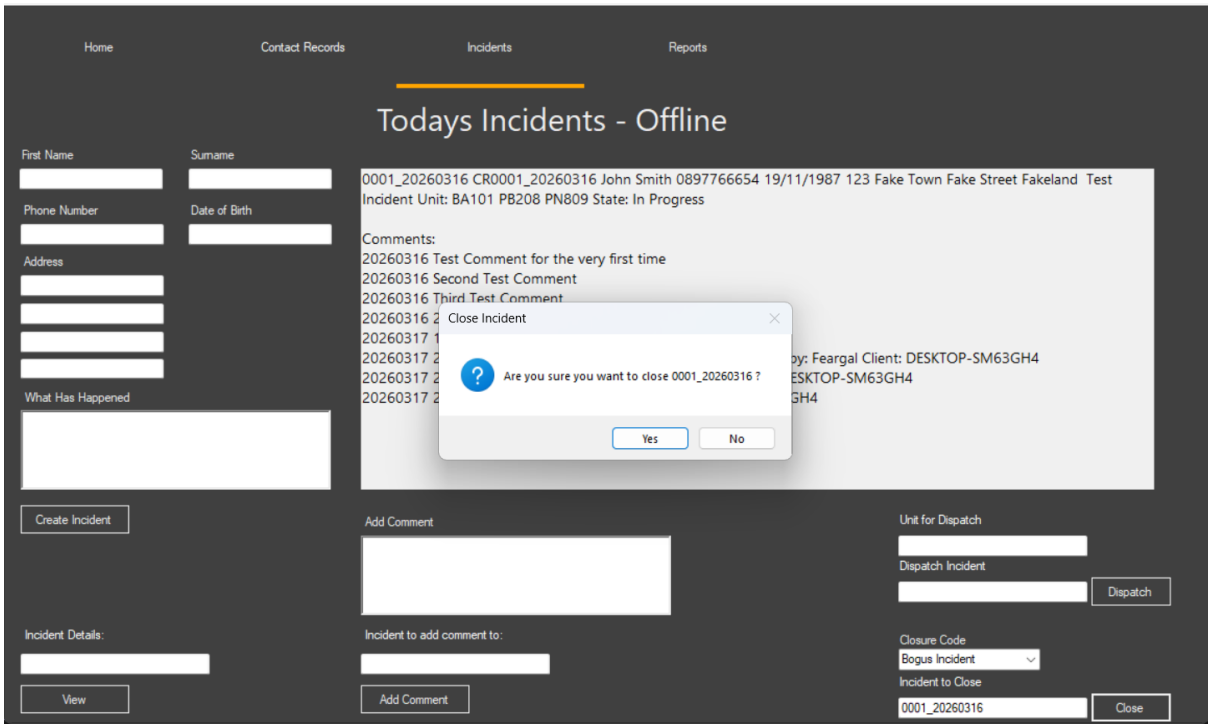

Figure 16 - Utilising Closure Codes and MessageBoxes to ensure data is updated correctly

```
// If the incident exists - give a new value to update or return info to the user depending on the state value of the Incident
    if (testValue == find)
    {
        var closeIncident = MessageBox.Show("Are you sure you want to close " + find + " ?", "Close Incident", MessageBoxButtons.YesNo, MessageBoxI
        if (closeIncident == DialogResult.Yes)
        {
            if (filtered.Contains(" State: New") && closureCode == "RWD")
            {
                string update = filtered.Replace(" State: New", " Unit: Closure Code: RWD State: Closed");
                updated = all.Replace(filtered, update);
            }
            else if (filtered.Contains(" State: New") && closureCode != "RWD")
            {
                MessageBox.Show("A 'New' incident can either be dispatched or closed by RWD only");
                closureCodeBox.SelectedItem = "-";
                incidentToCloseTextBox.Clear();
                reader1.Close();
                return;
            }
            else if (filtered.Contains(" State: In Progress") && closureCode != "RWD")
            {
                string update = filtered.Replace(" State: In Progress", " Closure Code: " + closureCode + " State: Closed");
                updated = all.Replace(filtered, update);
            }
```

Figure 17 - Code to validate if the user wished to progress depending on the result from a MessageBox

There was a different reaction from the users when I presented them with the Windows Forms version of ECADS. There was an energy from them that was positive, I was faced with

questions on request what I could do to improve it. As a result of the feedback and questions I implemented the following additions:

- Selecting a Unit to Dispatch (This was in the Bash and PowerShell version but only applied when closing the Incident – by implementing this, users could more easily manage Incidents In Progress)
- Highlighting the Contact Record and Incident number in the Display Box

Other feedback received was regarding the size of the display box in each of the views. Initially the users thought it was too large and was taking up too much space. When I asked them to create multiple entities as if they were really using it to work and review comments they agreed that it was the correct size, as more Contact Records and Incidents were created, the size mattered in reviewing and retrieving data.

Meeting Requirements

Table 2 - Functionality meeting Requirements across all three iterations

	Bash	**PowerShell**	**Windows Forms**
Req. 1	Bash Script can be run locally on a client	PS Script can be run locally on clients (all clients in the Production environment are Windows clients)	ECADS.exe can be run locally on all production clients with standard privileges
Req. 2	Possible to add shortcut to users Desktop by Group Policy	Possible to add shortcut to users Desktop by Group Policy	Possible to add shortcut to users Desktop by Group Policy

Req. 3	Bash script flows with simple menus that users must choose from – feedback from UAT was positive	PS script is similar to the Bash version, except it has the addition of colour which users during UAT were very positive about	The graphical version of the application was reported as simple to follow and easy to interact with
Req. 4	All users in UAT reported that the menu driven aspect of the script was easy to engage with, although repetitive	All users in UAT reported that the menu driven aspect of the script was easy to engage with, although repetitive – colours made the different sections stand out more, particularly effective for data validation	Reports that the system was intuitive and simple to follow – users can see clearly what they are entering and feedback is provided to the end user by means of message pop-up boxes
Req. 5	Script launches without escalated privileges – no log in required, actions by users are recorded in files with the logged in username of the client for that session	Script launches without escalated privileges – no log in required, actions by users are recorded in files with the logged in username of the client for that session	Application launches without escalated privileges – no log in required, actions by users are recorded in files with the logged in username of the client for that session
Req. 6	End user is provided the option to choose to	End user is provided the option to choose to	The end user can choose to create a Contact Record

	create an Incident each time they create a Contact Record – if the user chooses to create an Incident from a Contact Record, the Contact Record Number is recorded on the Incident	create an Incident each time they create a Contact Record – if the user chooses to create an Incident from a Contact Record, the Contact Record Number is recorded on the Incident	without an incident being created (similar to the current application) or can choose to create a Contact Record and simultaneously create an Incident by the use of Contact Record Closure Codes – if the user chooses to create an Incident from a Contact Record, the Contact Record Number is recorded on the Incident
Req. 7	Comments can be added to all active Incidents (i.e. Incidents that do not have a state of "Closed")	Comments can be added to all active Incidents (i.e. Incidents that do not have a state of "Closed")	Comments can be added to all active Incidents (i.e. Incidents that do not have a state of "Closed")
Req. 8	Closed Contact Records and Incidents can be recalled by the user to view their details but can no longer be updated	Closed Contact Records and Incidents can be recalled by the user to view their details but can no longer be updated	Closed Contact Records and Incidents can be recalled by the user to view their details but can no longer be updated

Req. 9	All Contact Records and Incidents that are created can be recalled by the user to view their details	All Contact Records and Incidents that are created can be recalled by the user to view their details	All Contact Records and Incidents that are created can be recalled by the user to view their details
Req. 10	** As of yet to be fully developed – The .txt files that are generated can be extracted and using a BCP can be imported into an SQL staging table for syncing with the main application **	** As of yet to be fully developed – The .txt files that are generated can be extracted and using a BCP can be imported into an SQL staging table for syncing with the main application **	** As of yet to be fully developed – The .txt files that are generated can be extracted and using a BCP can be imported into an SQL staging table for syncing with the main application **
Req. 11	No report functionality developed for Bash Script	No report functionality developed for the PS Script	Limited reporting functionality developed – i.e. listing the number of Contact Records and Incidents that were created on a certain date

Project Specific Considerations

Data Validation

When considering the use of this application, the data that is input is of the utmost importance. To account for that there were various checks input in each iteration of the system to assist users in correct data input and avoiding the well-known saying of “rubbish in, rubbish out”. These included only allowing numerical values for the telephone number in the Bash script and PowerShell script versions and including mandatory fields in the Windows Forms application and leveraging other input values before allowing the user to progress further or create an entity.

Testing

Throughout the development of this project, I engaged in testing the applications myself. This was usually undertaken after each functional change was introduced to the project. These tests, although informal, took the form of taking an aspect of functionality, understanding the expected result and comparing that with the actual result. This often resulted in finding that the new functionality worked as intended, but that it now meant prior code had to be refactored to account for the change. This was not an issue during the development phase of this project as I viewed it as something necessary to ensure functional requirements were met and that I could further progress with development of other functional aspects of the project.

Apart from my own individual testing, I engaged past and current users in UAT. This proved invaluable. It first of all gave me the reassurance that I was creating something that was viable and others could see themselves using it, but it also spurred me on to make an application worthwhile, knowing the importance of this data capture and recall.

The basic Create, Read, Update and Delete functions of any data driven application were tested (all bar Delete as it is explicitly disallowed from the system) and these passed all tests. One major advantage of Microsoft Visual Studio is that it provides some system monitoring when running the application in debug mode. As I tested the Windows Forms version of the application I could monitor the Process Memory and CPU percentage of all processors. The Process Memory never exceeded 24MB and the CPU percentage remained at 0% after an initial 4% at start up.

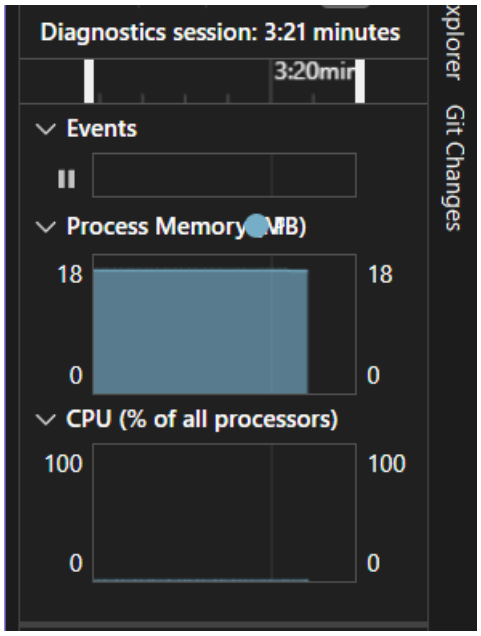

Figure 18 - Diagnostic capture when running Windows Forms version of ECADS

Use of AI

I did not use AI in any part of this project. On reflection, this may have put me at a disadvantage, however a major reason I chose this project was to further my understanding of programming and the C# language. I feel that if I had engaged the use of AI that I would not have been able to develop the programming and critical thinking skills that have increased as a result of this project. It is true, I could have queried some AI tool to assist with faster development or to address some of the challenges I encountered, but I feel that would have subverted the learning

process that is inherent to a project such as this and also I feel I would not have the intimate knowledge that I do have of each iteration.

I can see in future or for further development of this application that I could engage the use of AI. The areas I feel that it may be in a position to assist is reviewing the code I have input and perhaps making it more efficient. I took the approach of learning programming languages and making functional requirements possible, but that does not necessarily mean I did that in the correct or most efficient manner. As I now have an understanding of the C# language and how it operates, I think going forward it would be easier to engage with AI and to have a better understanding of AI generated C# code.

Conclusion

Personal Reflection

Looking back over this project as a hole I feel I have achieved what I set out to create. I have three separate iterations of an application which can be deployed to local clients and can capture relevant data in an offline environment. As I reflect, I believe that this project could have easily been undertaken as some form of a webapp incorporating web forms and javascript to provide the same end user feel. I think that overlooks part of the purpose of this project for me however, I have gained new insight into two new languages that I have not been instructed on, one which I had zero involvement with previously. Although I work as an administrator on the GardaSAFE system and this involves configuration work, that work is not reflective of the C# language, something I discovered very quickly!

There were certain areas that proved challenging and frustrating – I found writing the Bash script to be the most enjoyable, I just found the syntax easy and engaging, whereas I thought the PowerShell was overly verbose to achieve the same functionality. C#, although perhaps not

as popular as it once was, brought me though a different way of thinking. The application and code is somewhat event based and from this I found that the vast majority of my code was based on IF statements. I had read that this is a sure sign of a novice developer and I cannot argue with that, they seemed the most logical way to achieve what I wanted to.

What I found most valuable during this project was the problem solving challenges – all of which were raised and imposed on me by myself! I wanted to achieve something specific and I was able to, by to be able to I needed to think differently and to come at the problem from multiple different aspects. I actually learned the concept of a method during this project and it was towards the end! I wanted to highlight the Contact Record number and the Incident number when they were being displayed in their respective rich text display box in the Home view, the Contact Record view and the Incident view. I developed the code to achieve this and I then implemented this in each of the views. I discovered that I was just re-typing the same code to be executed depending on the event. I put the code into a simple method without an argument and this worked fine until I tried to implement it in the Home view, the Contact Record number and Incident number are different string lengths. It was at this point I realised that I could include an int argument in my method to make it applicable to whichever event was being triggered.

```
// Method to make the first number of chars of either a Contact Record or Incident bold when displayed in the Display Box
2 references
private void Emphasis_DisplayBoxSearch(int length)
{
    for (int i = 0; i < searchDisplayBox.Text.Length; i++)
    {
        if (searchDisplayBox.Text[i] == '\n')
        {
            int selectIndex = i + 1;
            int[] bold = { 0, selectIndex };
            for (int b = 0; b < bold.Length; b++)
            {
                int emphasis = bold[b];
                searchDisplayBox.Select(emphasis, length);
                searchDisplayBox.SelectionFont = new Font(searchDisplayBox.Font, FontStyle.Bold);
            }
        }
    }
}
```

Figure 19 - Method to apply bold on Contact Record or Incident numbers in Display box

Areas for further development

Reports

Unfortunately, although I was able to implement some form of reports for supervisors in the Windows Forms version of the application, I did not implement any reporting mechanism in the Bash or PowerShell versions. This is an area that I would like to improve upon in future. Time constraints meant that I was unable to give this the full attention it deserved but how I could see this being developed is to provide more information on the number of Contact Records and Incidents created by a certain user (leveraging the user name in the details of the specific entity, extracting it from the data using a regex and providing the number back to the user). I also think a button to generate a .txt file with the report results would be helpful, this would allow users to fully implement governance and auditing of use of the system.

Merging Data with SQL

One aspect I was unable to implement during this project was merging the data captured with the live database. I have proposed how this could be achieved in the body of this report, however this is still theoretical and requires further development. Once the staging table version of this is working I would hope to further develop this out to utilise the REST Api that is part of the main system and bypass having to use BCP to inject the data into SQL.

Windows Forms Application Design

I would like to review how the tabs operate. I have implemented a basic tab layout using buttons but I feel that if I had used the tabs content within Windows Forms that I could implement other functionality, such as opening specific tabs based on different user events.

Windows Forms - Rich Text Box

I included a Rich Text Box to display the data read from the .txt files back to the user. The rich text box allowed me to display the data and also format it, in this project I emboldened the Contact Record number and the Incident number to allow easier identification for the end user. I would like to revisit this in future and perhaps replace it with a data grid view. I think by introducing the data grid view it would make it easier for users to select the Incident or Contact Record they want to engage with by clicking on it. This would allow updating or adding comments become easier by clicking instead of typing in the Incident number to add the comment. This would also help with merging to SQL I believe.

Final Thoughts

Overall I found the project a very enjoyable experience and certainly found it a capstone to my learning for the HDip in Computer Science. I would like to take this opportunity to thank my Supervisor Dr. Sinead O'Neill for her engagement and guidance throughout my project development and also to Mr. Colm Dunphy and all of the staff in the Faculty of Science and Computing at SETU.

References

Atlassian. (2026) *What is Agile?* Available at: https://www.atlassian.com/agile (Accessed: 27 March 2026).

Garda Inspectorate. (2014) *Report of the Garda Síochána Inspectorate: Crime Investigation October 2014.* Available at: https://www.gsinsp.ie/wp-content/uploads/2019/07/Crime-Investigation-Full-Report.pdf (Accessed: 15 February 2026).

McCarthaigh, S. (2021) *€13.5m for new Garda IT system to record contact with public.* Available at: https://www.irishexaminer.com/news/courtandcrime/arid-40248977.html (Accessed: 15 February 2026).

Microsoft Learn. (2026.a) *Import and export bulk data using bcp (SQL Server)* Available at: https://learn.microsoft.com/en-us/sql/relational-databases/import-export/import-and-export-bulk-data-by-using-the-bcp-utility-sql-server?view=sql-server-ver17 (Accessed 27 March 2026)

Microsoft Learn. (2026.b) *Overview of .NET Framework.* Available at: https://learn.microsoft.com/en-us/dotnet/framework/get-started/overview (Accessed 27 March 2026).

SAAB AB. (2020) *Irish National Police selects Saab's SAFE Command & Control Solution*. Available at: https://www.saab.com/newsroom/press-releases/2020/irish-national-police-selects-saabs-safe-command--control-solution (Accessed: 15 February 2026).

SAAB AB. (2025) *SAAB PSS – SAAB UK monthly update – October 2025 v4*

Corporate slide deck.

Appendix 1 – Challenge Solutions

PowerShell

Powershell menu

https://www.business.com/articles/powershell-interactive-menu/ - accessed 25/02/2026

Powershell readhost array

https://stackoverflow.com/questions/50354484/creating-array-elements-through-read-host-how-can-i-move-an-element-to-another

C#

To get the array to work to remove "State: Closed"

https://stackoverflow.com/questions/238002/replace-line-breaks-in-a-string-c-sharp

https://stackoverflow.com/questions/6466688/search-specific-string-and-return-whole-line

https://www.geeksforgeeks.org/c-sharp/string-join-method-c-sharp/

Validate the Incident number for closing/dispatching

https://stackoverflow.com/questions/27825015/c-sharp-efficiently-check-if-string-at-specific-position-some

Message box validation close incident

https://www.jquery-az.com/c-sharp-messagebox/

Timestamp:

https://stackoverflow.com/questions/69576227/datetime-utcnow-tostringyyyy-mm-dd-hhmmss-gives-different-formats-for-diff

Count file lines:

https://www.tutorialspoint.com/article/chash-program-to-count-the-number-of-lines-in-a-file